

Informe de Laboratorio 4

Interrupciones y Rutinas de Servicio en MIPS32

Samuel Cordero
C.I V-31.678.592
Dervis Martínez
C.I V-31.456
Universidad de Carabobo

Facultad de Ciencias y Tecnología (FACYT)
Departamento de Computación
Arquitectura del Computador
Julio 2025

1. Ciclo de Interrupción de Hardware

El ciclo de interrupción de hardware sigue una secuencia específica desde que ocurre el evento hasta que se atiende:

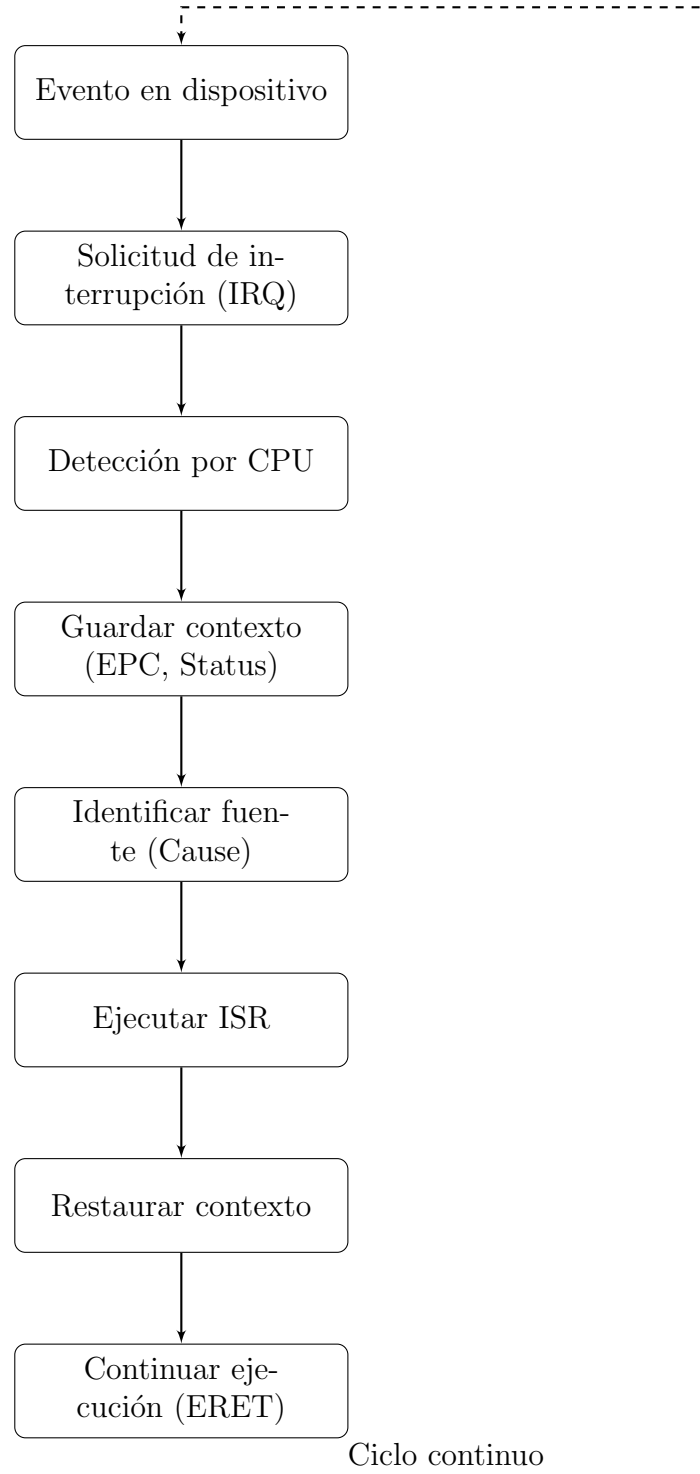
1. **Ocurrencia del evento:** Un dispositivo periférico (teclado, temporizador, etc.) detecta un evento que requiere atención.
2. **Solicitud de interrupción (IRQ):** El dispositivo activa la línea de interrupción correspondiente.
3. **Detección por la CPU:** Al final del ciclo de instrucción, la CPU verifica si hay interrupciones pendientes.
4. **Habilitación de interrupciones:** Si las interrupciones están habilitadas (bit IE en registro Status):
 - Completa la instrucción actual
 - Guarda el contexto (PC en EPC, copia Status en registro temporal)
 - Deshabilita interrupciones (bit IE = 0)
5. **Identificación de la fuente:** Consulta el registro Cause para determinar qué dispositivo generó la interrupción.
6. **Salto a la rutina de servicio:** Carga la dirección base del vector de interrupciones (0x80000180 en MIPS) en PC.
7. **Ejecución de ISR:**
 - Guarda registros en pila
 - Atiende el dispositivo

- Limpia la condición de interrupción

8. **Recuperación del contexto:** Restaura registros desde la pila.

9. **Retorno:** Ejecuta instrucción ERET que:

- Restaura el registro Status
- Salta a la dirección guardada en EPC
- Reactiva interrupciones



2. Diferencias entre Gestión de E/S con Sondeo e Interrupciones

Comparación detallada

Mecanismo de operación

- **Sondeo:** La CPU **inicia** la comunicación verificando activamente el estado del dispositivo en intervalos regulares. Este enfoque es **orientado a la CPU** y consume recursos constantemente.
- **Interrupciones:** El dispositivo **inicia** la comunicación notificando a la CPU cuando necesita atención. Este enfoque es **orientado al dispositivo** y permite a la CPU trabajar en otras tareas hasta la notificación.

Eficiencia de CPU

- **Sondeo:** Utiliza ciclos de CPU incluso cuando no hay operaciones pendientes. En el peor caso (dispositivo lento), la CPU puede estar inactiva ¿90 % del tiempo en espera activa.
- **Interrupciones:** Permite máximo aprovechamiento de la CPU. Solo consume ciclos cuando realmente hay que atender un dispositivo. En aplicaciones típicas, reduce el uso de CPU en E/S en un 70-90 %.

Tiempo de respuesta

- **Sondeo:** Latencia determinada por el intervalo de sondeo. Para garantizar respuesta rápida se requiere alta frecuencia de sondeo, lo que aumenta el consumo de recursos.
- **Interrupciones:** Latencia mínima constante. El tiempo entre el evento y el inicio de la ISR es fijo y típicamente pequeño (10-100 ciclos en MIPS).

Implementación en sistemas MIPS

- **Sondeo:** No requiere configuración especial de registros de control. Solo necesita acceso directo a registros de estado de dispositivos.
- **Interrupciones:** Requiere configuración de registros especiales:
 - Habilitación global (bit 0 de Status)
 - Habilitación por dispositivo (bits 8-15 de Status)
 - Manejo de EPC y Cause
 - Implementación de rutinas de servicio (.ktext)

Casos de uso típicos

- **Sondeo:** Adecuado para sistemas simples con un solo dispositivo, o cuando la frecuencia de eventos es muy alta (mayor que la capacidad de manejo de interrupciones).
- **Interrupciones:** Esencial para sistemas complejos con múltiples dispositivos, sistemas en tiempo real, y aplicaciones donde la eficiencia energética es crítica.

Ejemplo práctico en `buffer.asm`

El programa `buffer.asm` utiliza interrupciones para:

- Atender entrada de teclado solo cuando ocurre una pulsación (no verifica constantemente)
- Activar el temporizador para los intervalos de 20 segundos
- Combinar ambas interrupciones sin conflicto

Implementar esto con sondeo requeriría:

```
1 main_loop:
2     # Verificar temporizador
3     lw $t0, TIMER_STATUS
4     andi $t0, $t0, 1
5     bnez $t0, handle_timer
6
7     # Verificar teclado
8     lw $t0, KEYBOARD_STATUS
9     andi $t0, $t0, 1
10    bnez $t0, handle_key
11
12    j main_loop
```

Este enfoque consumiría 100 % del CPU aunque no hubiera actividad.

Ejemplo en semáforo.asm

El semáforo utiliza interrupciones para:

- Detectar pulsaciones del botón (tecla 's') sin verificación constante
- Manejar múltiples temporizadores para transiciones de estado
- Combinar eventos de teclado y temporizador eficientemente

Conclusiones

La gestión por interrupciones es superior en la mayoría de escenarios modernos debido a su eficiencia y escalabilidad. El sondeo solo se justifica en sistemas muy simples o cuando la frecuencia de eventos es extremadamente alta. En los programas desarrollados (buffer.asm y semáforo.asm), el uso de interrupciones permite una gestión eficiente de múltiples dispositivos y eventos simultáneos con mínimo consumo de recursos.

3. Ventajas del Uso de Interrupciones en Términos de Uso del Procesador

1. **Maximización de trabajo útil:** La CPU no desperdicia ciclos verificando dispositivos inactivos
2. **Paralelismo efectivo:** Permite solapamiento de cómputo y operaciones de E/S
3. **Eficiencia energética:** Permite estados de bajo consumo (sleep) durante esperas
4. **Multitarea:** Soporte para múltiples procesos con tiempos de respuesta garantizados
5. **Latencia controlada:** Respuesta inmediata a eventos críticos sin retrasos por sondeo
6. **Throughput mejorado:** Mayor cantidad de trabajo completado por unidad de tiempo

4. Registros Especiales para Gestión de Interrupciones en MIPS32

5. Necesidad de Guardar el Contexto en Rutinas de Servicio

El guardado de contexto es esencial porque:

- **Transparencia:** El programa interrumpido debe continuar como si nada hubiera ocurrido
- **Consistencia de estado:** Registros contienen datos críticos para la ejecución correcta
- **Prevención de corrupción:** La ISR usa registros que podrían estar en uso por el programa principal
- **Multi-anidamiento:** Soporte para interrupciones durante el manejo de otras interrupciones

Consecuencias de no guardar contexto:

- Corrupción de datos en registros
- Comportamiento impredecible del programa
- Fallos en cascada
- Bloqueos del sistema

6. Excepciones en Sistemas MIPS32

a) Situaciones generadoras de excepciones

1. Desbordamiento aritmético (overflow)
2. Instrucción ilegal (código de operación no reconocido)
3. Fallo de página (dirección virtual sin mapeo físico)
4. Violación de privilegios (instrucción privilegiada en modo usuario)
5. Acceso a memoria no alineado (LW/SW en dirección no múltiplo de 4)
6. Llamada al sistema (SYSCALL)
7. Breakpoint (BREAK)
8. Fallo de bus (acceso a dirección física inexistente)

b) Etapas del pipeline que provocan excepciones

7. Estrategias para Tratamiento de Excepciones e Interrupciones

a) Diferencias entre interrupciones y excepciones

b) Estrategias de tratamiento

1. Vectorizado:

- Diferentes excepciones saltan a distintas direcciones
- Implementado mediante tabla de vectores
- Más rápido para manejo específico

2. No vectorizado:

- Todas las excepciones saltan a misma dirección (0x80000180)
- Software determina causa consultando registro Cause
- Implementado en MIPS estándar

Función del registro EPC:

- Almacena dirección de la instrucción que causó la excepción
- Permite reanudar ejecución después de manejar excepción
- Instrucción ERET restaura PC desde EPC

8. Habilitación de Interrupciones

a) Procedimiento de habilitación

Teclado:

```
1 li $v0, 35      # Habilitar interrupciones de teclado
2 syscall
```

Pantalla:

```
1 li $v0, 36      # Habilitar interrupciones de pantalla
2 syscall
```

Procesador (registro Status):

```
1 mfc0 $t0, $12    # Leer registro Status
2 ori $t0, 0x1     # Habilitar globalmente (bit 0)
3 ori $t0, 0x800   # Habilitar teclado (bit 11)
4 ori $t0, 0x1000  # Habilitar temporizador (bit 12)
5 mtc0 $t0, $12    # Escribir registro Status
```

b) Consecuencias de habilitar dispositivos pero no el procesador

- Las IRQ se registran en registro Cause (bits pendientes)
- CPU ignora completamente las solicitudes
- No se ejecuta ninguna rutina de servicio
- Dispositivos pueden quedar en estado bloqueado
- Eventos críticos no se atienden
- Posible pérdida de datos (buffers sobrescritos)

9. Procesamiento de Interrupciones de Reloj

a) Paso a paso

1. **Evento:** Temporizador alcanza valor programado
2. **Solicitud:** Módulo de temporización activa línea IRQ
3. **Detección:** CPU detecta IRQ al final de ciclo de instrucción
4. **Guardado hardware:**
 - $PC \rightarrow EPC$
 - $Status \rightarrow$ registro temporal
 - $IE = 0$ (deshabilita interrupciones)
 - $EXL = 1$ (entra en nivel excepción)
5. **Salto:** $PC \leftarrow 0x80000180$
6. **Guardado software:** Rutina salva registros en pila
7. **Identificación:** Lee registro Cause para confirmar IRQ de reloj
8. **Procesamiento:**
 - Actualizar contadores
 - Planificar cambio de contexto
 - Reprogramar temporizador
9. **Limpieza:** Resetear flag de interrupción
10. **Restauración:** Recupera registros desde pila
11. **Retorno:** Ejecuta ERET $\rightarrow$ restaura Status y PC, reactiva interrupciones

b) Importancia de guardar el contexto

- Garantiza que programa interrumpido continúe correctamente
- Previene que ISR corrompa estado de aplicación
- Permite múltiples interrupciones anidadas
- Soporta rutinas compartidas por diferentes interrupciones

10. Interrupciones de Reloj para Control de Ejecución

a) Aplicaciones

1. **Prevenir bucles infinitos:**
 - Programar temporizador con límite de tiempo
 - En ISR: terminar proceso o tomar acción correctiva

```
1 # Configurar temporizador
2 li $a0, MAX_TIME
3 li $v0, 33
4 syscall
```

2. Finalizar programas que superan tiempo máximo:

- Esencial en sistemas de tiempo real
- ISR fuerza finalización si se excede tiempo

b) Manejo de programa que finaliza antes

- Deshabilitar interrupción: Cancelar temporizador pendiente
- Reiniciar temporizador: Para siguiente intervalo
- Liberar recursos: Memoria, dispositivos asociados
- Notificar sistema: Registrar finalización exitosa

11. Análisis y Discusión de Resultados

a) Buffer de Entrada de Teclado (buffer.asm)

Implementación:

- Buffer circular de 100 caracteres
- Head y tail para gestión FIFO
- Interrupciones de teclado y temporizador

Logros:

- Manejo eficiente de entrada asíncrona
- Sin pérdida de datos durante impresión
- Uso óptimo de CPU durante esperas

Mejoras potenciales:

- Implementar lock para acceso concurrente
- Añadir manejo de buffer lleno
- Soporte para teclas especiales (Backspace)

b) Semáforo con Pulsador (semaforo.asm)

Implementación:

- Máquina de estados: Verde $\rightarrow$ Amarillo $\rightarrow$ Rojo
- Control por pulsador (tecla 's')
- Temporizadores para transiciones

Logros:

- Respuesta inmediata a evento externo
- Transiciones temporizadas precisas
- Simulación completa de comportamiento real

Limitaciones:

- No manejo de pulsaciones múltiples
- Sin prioridad para eventos concurrentes
- Estados bloqueantes durante temporizaciones

c) Conclusiones Generales

- Las interrupciones permiten construir sistemas reactivos eficientes
- MIPS provee mecanismos robustos para gestión de eventos
- El diseño modular facilita expansión y mantenimiento
- La gestión adecuada de contexto es crítica para estabilidad
- Los temporizadores son esenciales para sistemas en tiempo real
- La combinación de interrupciones de hardware y temporizadores permite implementar sistemas complejos con comportamiento determinista

Sondeo
(Polling)

- Ventajas:
- Implementación simple

■ Comportamiento predecible

■ Fácil depuración
- Ventajas:
- CPU libre para otras tareas

■ Baja consumo energético

■ Respuesta inmediata a eventos

■ Mayor eficiencia de múltiples dispositivos
- Desventajas:
- Consumo

Interf...

- Ent...

■ Me...

■ Im...

5 ISR

6

Venta...

- CP...

■ Baj...

■ Res...

■ Ma...
- Desve...
- Con...

Registro	CP0	Función
Status	Reg 12	■ Bit 0 (IE): Habilitación global de interrupciones ■ Bits 8-15 (IM): Máscara para habilitar dispositivos específicos ■ Bit 1 (EXL): Nivel de excepción (1 = en excepción)
Cause	Reg 13	■ Bits 8-15 (IP): Interrupciones pendientes ■ Bits 2-6 (ExcCode): Código de excepción
EPC	Reg 14	Almacena la dirección de retorno después de atender interrupción
BadVAddr	Reg 8	Dirección inválida en fallos de memoria
Context	Reg 4	Información para manejo de fallos de página

Etapas	Excepciones
IF (Instruction Fetch)	■ Fallo de página en acceso a instrucción ■ Dirección de instrucción inválida
ID (Instruction Decode)	■ Instrucción ilegal ■ Violación de privilegios ■ Instrucción no implementada
EX (Execute)	■ Desbordamiento aritmético ■ División por cero
MEM (Memory Access)	■ Fallo de página en acceso a datos ■ Dirección de memoria no alineada ■ Fallo de bus

Característica	Interrupciones	Excepciones
Origen	Eventos externos (dispositivos)	Eventos internos (ejecución)
Sincronía	Asíncronas (independientes de instrucción)	Síncronas (relacionadas con instrucción)
Predictibilidad	Ocurren en cualquier momento	Ocurren durante ejecución específica
Ejemplos	Teclado, temporizador	Overflow, página inválida